Lab Task – 1

Task – 1

1. Print a string "Assembly Language".

```
.model small                ; Specify a small memory model
.stack                      ; Define the stack segment
.data
m db "Assembly Language$"   ; Define a string with a '$' terminator for output

.code
main proc                   ; Start of the main procedure

    mov ax, @data           ; Load the address of the data segment
    mov ds, ax              ; Initialize the data segment register

    mov ah, 9               ; DOS function to display a string
    lea dx, m               ; Load the address of the string into DX
    int 21h                 ; Call interrupt to display the string

Exit:
    mov ah, 4ch             ; DOS function to exit the program
    int 21h                 ; Call interrupt to terminate

    main endp               ; End of the main procedure
end main                    ; Mark the end of the program
```

Task – 2

2. Show one user input and one predefined input.

```
.model small                                    ; Specify the small memory model
.stack 100h                                        ; Reserve 256 bytes for the stack
.data
msg db "Predefine Input $"                      ; Define a message with a '$' terminator for display

.code
main proc                   ; Start of the main procedure

    mov ax, @data           ; Load the address of the data segment
    mov ds, ax              ; Initialize the data segment register

    mov ah, 9               ; DOS function to display a string
    lea dx, msg             ; Load the address of the string into DX
    int 21h                 ; Call interrupt to display the string

    mov ah, 1               ; DOS function to read a single character
    int 21h                 ; Call interrupt to get input
    mov bl, al              ; Store the input character in BL

    mov ah, 2               ; DOS function to display a single character
    mov dl, bl              ; Load the character from BL into DL
    int 21h                 ; Call interrupt to display the character

exit:
    mov ah, 4ch             ; DOS function to terminate the program
    int 21h                 ; Call interrupt to exit
```

```
    main endp                ; End of the main procedure
end main                     ; Mark the end of the program
```

Task – 3

```
3. Read two character and display it new line.
       Sample Output:
              AB
              AB
```

```
.model small                 ; Specify small memory model
.stack 100h                  ; Reserve 256 bytes for the stack
.code
main proc                    ; Start of the main procedure

    mov ah, 1                ; DOS function to read a single character
    int 21h                  ; Call interrupt to read first character
    mov bl, al               ; Store the first character in BL

    mov ah, 1                ; DOS function to read a single character
    int 21h                  ; Call interrupt to read second character
    mov bh, al               ; Store the second character in BH

    mov ah, 2                ; DOS function to display a single character
    mov dl, 10               ; Line feed (new line)
    int 21h                  ; Call interrupt to display newline
    mov dl, 13               ; Carriage return
    int 21h                  ; Call interrupt to display carriage return

    mov ah, 2                ; DOS function to display a single character
    mov dl, bl               ; Load the first character (stored in BL) into DL
    int 21h                  ; Call interrupt to display the first character

    mov ah, 2                ; DOS function to display a single character
    mov dl, bh               ; Load the second character (stored in BH) into DL
    int 21h                  ; Call interrupt to display the second character

    mov ah, 2                ; DOS function to display a single character
    mov dl, 10               ; Line feed (new line)
    int 21h                  ; Call interrupt to display newline
    mov dl, 13               ; Carriage return
    int 21h                  ; Call interrupt to display carriage return

exit:
    mov ah, 4ch              ; DOS function to terminate the program
    int 21h                  ; Call interrupt to exit

    main endp                ; End of the main procedure
end main                     ; Mark the end of the program
```

Task – 4

```
4. Read a character A and display it A to Z times on the next line.
   Hints: use inc instructions
        Sample output:
               A to Z.
```

```
.model small                 ; Specify small memory model
.stack 100H                  ; Reserve 256 bytes for the stack
.data                        ; Data segment (not used here)
.code
main proc                    ; Start of the main procedure

    mov ax, @data            ; Load the address of the data segment
```

```
    mov ds, ax              ; Initialize the data segment register

    mov cx, 26              ; Set the loop counter to 26 (for 26 alphabets)
    mov ah, 2               ; DOS function to display a single character
    mov dl, 65              ; ASCII value of 'A'

level1:
    int 21h                 ; Call interrupt to display the current character
    inc dl                  ; Increment DL to the next ASCII character
    loop level1             ; Decrement CX and repeat until CX = 0

Exit:
    mov ah, 4ch             ; DOS function to terminate the program
    int 21h                 ; Call interrupt to exit

    main endp               ; End of the main procedure
end main                    ; Mark the end of the program
```

Lab Task – 2

Task – 1

```
.model small                ; Use small memory model
.stack 100h                 ; Reserve 256 bytes for the stack
.data
a db 'Input First Number: $'    ; Prompt for the first number
b db 'Input Second Number: $'   ; Prompt for the second number
output db 'Result: $'           ; Message to display the result

.code
main proc

    mov ax, @data           ; Load the address of the data segment
    mov ds, ax              ; Initialize the data segment register

    ; Display the first prompt
    mov ah, 9               ; DOS function to display a string
    lea dx, a               ; Load the address of the string 'a'
    int 21h                 ; Call interrupt to display the string

    ; Input the first number
    mov ah, 1               ; DOS function to read a single character
    int 21h                 ; Call interrupt to read input
    sub al, '0'             ; Convert ASCII to numeric
    mov bl, al              ; Store the first number in BL

    ; Formatting: New line
    mov ah, 2               ; DOS function to display a character
    mov dl, 10              ; ASCII code for line feed (new line)
    int 21h                 ; Display line feed
    mov dl, 13              ; ASCII code for carriage return
    int 21h                 ; Display carriage return
```

```
; Display the second prompt
mov ah, 9               ; DOS function to display a string
lea dx, b               ; Load the address of the string 'b'
int 21h                 ; Call interrupt to display the string

; Input the second number
mov ah, 1               ; DOS function to read a single character
int 21h                 ; Call interrupt to read input
sub al, '0'             ; Convert ASCII to numeric

; Calculate the difference
sub bl, al              ; Subtract the second number from the first

; Formatting: New line
mov ah, 2               ; DOS function to display a character
mov dl, 10              ; Line feed
int 21h                 ; Display line feed
mov dl, 13              ; Carriage return
int 21h                 ; Display carriage return

; Display the result message
mov ah, 9               ; DOS function to display a string
lea dx, output          ; Load the address of the string 'output'
int 21h                 ; Call interrupt to display the string

; Display the result
add bl, '0'             ; Convert numeric result back to ASCII
mov dl, bl              ; Load the result into DL
mov ah, 2               ; DOS function to display a character
int 21h                 ; Call interrupt to display the result

exit:
mov ah, 4ch             ; DOS function to terminate the program
int 21h                 ; Call interrupt to exit
main endp               ; End of the main procedure
end main                ; Mark the end of the program
```

Task – 2

```
.model small                ; Use small memory model
.stack 100h                 ; Reserve 256 bytes for the stack
.data
a db 'Input First Number: $'    ; Prompt for the first number
b db 'Input Second Number: $'   ; Prompt for the second number
output db 'Result: $'           ; Message to display the result

.code
main proc

    mov ax, @data           ; Load the address of the data segment
    mov ds, ax              ; Initialize the data segment register
```

```
; Display the first prompt
mov ah, 9               ; DOS function to display a string
lea dx, a               ; Load the address of the string 'a'
int 21h                 ; Call interrupt to display the string

; Input the first number
mov ah, 1               ; DOS function to read a single character
int 21h                 ; Call interrupt to read input
sub al, '0'             ; Convert ASCII to numeric
mov bl, al              ; Store the first number in BL

; Formatting: New line
mov ah, 2               ; DOS function to display a character
mov dl, 10              ; ASCII code for line feed (new line)
int 21h                 ; Display line feed
mov dl, 13              ; ASCII code for carriage return
int 21h                 ; Display carriage return

; Display the second prompt
mov ah, 9               ; DOS function to display a string
lea dx, b               ; Load the address of the string 'b'
int 21h                 ; Call interrupt to display the string

; Input the second number
mov ah, 1               ; DOS function to read a single character
int 21h                 ; Call interrupt to read input
sub al, '0'             ; Convert ASCII to numeric

; Calculate the sum
add bl, al              ; Add the second number to the first

; Formatting: New line
mov ah, 2               ; DOS function to display a character
mov dl, 10              ; Line feed
int 21h                 ; Display line feed
mov dl, 13              ; Carriage return
    int 21h             ; Display carriage return

    ; Display the result message
    mov ah, 9           ; DOS function to display a string
    lea dx, output      ; Load the address of the string 'output'
    int 21h             ; Call interrupt to display the string

    ; Display the result
    add bl, '0'         ; Convert numeric result back to ASCII
    mov dl, bl          ; Load the result into DL
    mov ah, 2           ; DOS function to display a character
    int 21h             ; Call interrupt to display the result

exit:
    mov ah, 4ch         ; DOS function to terminate the program
    int 21h             ; Call interrupt to exit
```

```
    main endp                ; End of the main procedure
end main                     ; Mark the end of the program
```

Task – 3

```
.model small                 ; Use the small memory model
.stack 100h                  ; Reserve 256 bytes for the stack
.data
a db 'Input Lower case letter: $' ; Prompt for the lowercase letter
b db 'Upper case letter: $'       ; Prompt to display the uppercase letter

.code
main proc

    ; Initialize data segment
    mov ax, @data            ; Load the address of the data segment
    mov ds, ax               ; Initialize the data segment register

    ; Display the first prompt
    mov ah, 9                ; DOS function to display a string
    lea dx, a                ; Load the address of the string 'a'
    int 21h                  ; Call interrupt to display the string

    ; Input a lowercase letter
    mov ah, 1                ; DOS function to read a single character
    int 21h                  ; Call interrupt to read input
    mov bl, al               ; Store the input character in BL

    ; Formatting: New line
    mov ah, 2                ; DOS function to display a character
    mov dl, 10               ; ASCII code for line feed (new line)
    int 21h                  ; Display line feed
    mov dl, 13               ; ASCII code for carriage return
    int 21h                  ; Display carriage return

    ; Convert lowercase to uppercase
    sub bl, 32               ; Subtract 32 from the ASCII value to convert to uppercase

    ; Display the second prompt
    mov ah, 9                ; DOS function to display a string
    lea dx, b                ; Load the address of the string 'b'
    int 21h                  ; Call interrupt to display the string

    ; Display the converted uppercase letter
    mov dl, bl               ; Load the uppercase character into DL
    mov ah, 2                ; DOS function to display a character
    int 21h                  ; Call interrupt to display the character

    ; Exit program
    mov ah, 4Ch              ; DOS function to terminate the program
    int 21h                  ; Call interrupt to exit

exit:
```

```
    mov ah, 4ch             ; DOS function to terminate the program (redundant)
    int 21h
    main endp               ; End of the main procedure
end main                    ; Mark the end of the program
```

Task – 4

```
.model small                ; Use the small memory model
.stack 100h                 ; Reserve 256 bytes for the stack
.data
a db 'Input Upper case letter: $' ; Prompt for the uppercase letter
b db 'Lower case letter: $'       ; Prompt to display the lowercase letter

.code
main proc

    ; Initialize data segment
    mov ax, @data           ; Load the address of the data segment
    mov ds, ax              ; Initialize the data segment register

    ; Display the first prompt
    mov ah, 9               ; DOS function to display a string
    lea dx, a               ; Load the address of the string 'a'
    int 21h                 ; Call interrupt to display the string

    ; Input an uppercase letter
    mov ah, 1               ; DOS function to read a single character
    int 21h                 ; Call interrupt to read input
    mov bl, al              ; Store the input character in BL

    ; Formatting: New line
    mov ah, 2               ; DOS function to display a character
    mov dl, 10              ; ASCII code for line feed (new line)
    int 21h                 ; Display line feed
    mov dl, 13              ; ASCII code for carriage return
    int 21h                 ; Display carriage return

    ; Convert uppercase to lowercase
    add bl, 32              ; Add 32 to the ASCII value to convert to lowercase

    ; Display the second prompt
    mov ah, 9               ; DOS function to display a string
    lea dx, b               ; Load the address of the string 'b'
    int 21h                 ; Call interrupt to display the string

    ; Display the converted lowercase letter
    mov dl, bl              ; Load the lowercase character into DL
    mov ah, 2               ; DOS function to display a character
    int 21h                 ; Call interrupt to display the character

    ; Exit program
    mov ah, 4Ch             ; DOS function to terminate the program
    int 21h                 ; Call interrupt to exit
```

```
exit:
    mov ah, 4ch             ; DOS function to terminate the program (redundant)
    int 21h
    main endp               ; End of the main procedure
end main                    ; Mark the end of the program
```

Task – 5

```
.model small                ; Use the small memory model
.stack 100h                 ; Reserve 256 bytes for the stack
.data
a db 'Input First Number: $' ; Prompt for the first number
b db 'Input Second Number: $' ; Prompt for the second number
c db 'Input Third Number: $'  ; Prompt for the third number
msg db 'Biggest Number : $'   ; Prompt to display the result

.code
main proc

    ; Initialize data segment
    mov ax, @data           ; Load the address of the data segment
    mov ds, ax              ; Initialize the data segment register

    ; Display the first prompt
    mov ah, 9               ; DOS function to display a string
    lea dx, a               ; Load the address of the string 'a'
    int 21h                 ; Call interrupt to display the string

    ; Input the first number
    mov ah, 1               ; DOS function to read a single character
    int 21h                 ; Call interrupt to read input
    sub al, '0'             ; Convert ASCII to integer by subtracting '0'
    mov bl, al              ; Store the first number in BL

    ; Formatting: New line
    mov ah, 2               ; DOS function to display a character
    mov dl, 10              ; ASCII code for line feed (new line)
    int 21h                 ; Display line feed
    mov dl, 13              ; ASCII code for carriage return
    int 21h                 ; Display carriage return

    ; Display the second prompt
    mov ah, 9               ; DOS function to display a string
    lea dx, b               ; Load the address of the string 'b'
    int 21h                 ; Call interrupt to display the string

    ; Input the second number
    mov ah, 1               ; DOS function to read a single character
    int 21h                 ; Call interrupt to read input
    sub al, '0'             ; Convert ASCII to integer by subtracting '0'
    mov cl, al              ; Store the second number in CL
```

```
    ; Formatting: New line
    mov ah, 2               ; DOS function to display a character
    mov dl, 10              ; ASCII code for line feed (new line)
    int 21h                 ; Display line feed
    mov dl, 13              ; ASCII code for carriage return
    int 21h                 ; Display carriage return

    ; Compare the first number (BL) with the second number (CL)
    cmp bl, cl              ; Compare BL and CL
    jge greater             ; If BL >= CL, jump to 'greater'

    mov bl, cl              ; If CL > BL, move CL to BL

greater:
    ; Display the third prompt
    mov ah, 9               ; DOS function to display a string
    lea dx, c               ; Load the address of the string 'c'
    int 21h                 ; Call interrupt to display the string

    ; Input the third number
    mov ah, 1               ; DOS function to read a single character
    int 21h                 ; Call interrupt to read input
    sub al, '0'             ; Convert ASCII to integer by subtracting '0'
    mov dl, al              ; Store the third number in DL

    ; Compare the largest of the first two numbers (BL) with the third (DL)
    cmp bl, dl              ; Compare BL and DL
    jge result_ready        ; If BL >= DL, jump to 'result_ready'

    mov bl, dl              ; If DL > BL, move DL to BL

    ; Formatting: New line
    mov ah, 2               ; DOS function to display a character
    mov dl, 10              ; ASCII code for line feed (new line)
    int 21h                 ; Display line feed
    mov dl, 13              ; ASCII code for carriage return
    int 21h                 ; Display carriage return

result_ready:
    ; Display the result prompt
    mov ah, 9               ; DOS function to display a string
    lea dx, msg             ; Load the address of the string 'msg'
    int 21h                 ; Call interrupt to display the string

    ; Display the largest number
    add bl, '0'             ; Convert the integer back to ASCII
    mov dl, bl              ; Load the largest number into DL
    mov ah, 2               ; DOS function to display a character
    int 21h                 ; Call interrupt to display the character

    ; Exit the program
    mov ah, 4Ch             ; DOS function to terminate the program
```

```
    int 21h                 ; Call interrupt to exit

exit:
    mov ah, 4ch             ; DOS function to terminate the program (redundant)
    int 21h
    main endp               ; End of the main procedure
end main                    ; Mark the end of the program
```

Lab Task – 3

Task – 1

```
.model small                ; Use the small memory model
.stack 100H                 ; Reserve 256 bytes for the stack
.data                       ; Data segment (empty in this example)
.code

main proc
    ; Initialize data segment
    mov ax, @data           ; Load the address of the data segment
    mov ds, ax              ; Set the data segment register

    ; Initialize the value in AX
    mov ax, -4              ; Load -4 (signed value) into AX

    ; Compare AX with 0
    cmp ax, 0               ; Compare AX with 0
    jnl end_if              ; Jump to 'end_if' if AX >= 0 (JN = Jump if Not Less)

    ; If AX < 0, execute this block
    mov bx, -1              ; Set BX to -1

end_if:
    ; Exit the program
    mov ah, 4Ch             ; DOS function to terminate the program
    int 21h                 ; Call interrupt to exit

main endp                   ; End of the main procedure
end main                    ; Mark the end of the program
```

Task – 2

```
.model small                ; Use the small memory model
.stack 100H                 ; Reserve 256 bytes for the stack
.data                       ; Data segment (not used in this example)
.code

main proc
    ; Initialize data segment
    mov ax, @data           ; Load the address of the data segment
    mov ds, ax              ; Set the data segment register
```

```
    ; Load a signed value into AL
    mov al, -4              ; Load -4 (signed byte) into AL

    ; Compare AL with 0
    cmp al, 0               ; Compare AL with 0
    jnbe else_              ; Jump to 'else_' if AL > 0 (JNBE = Jump if Not Below or Equal)

    ; If AL <= 0, execute this block
    mov ah, 0FFh            ; Set AH to 255 (0xFF in hexadecimal)
    jmp exit                ; Jump to 'exit' to skip the 'else_' block

else_:
    ; If AL > 0, execute this block
    mov ah, 0               ; Set AH to 0

end_if:

exit:
    ; Exit the program
    mov ah, 4Ch             ; DOS function to terminate the program
    int 21h                 ; Call interrupt to exit

main endp                   ; End of the main procedure
end main                    ; Mark the end of the program
```

Task – 3

```
.model small                ; Use the small memory model
.stack 100h                 ; Reserve 256 bytes for the stack
.data                       ; Data segment (not used in this example)
.code

main proc
    ; Initialize data segment
    mov ax, @data           ; Load the address of the data segment
    mov ds, ax              ; Set the data segment register

    ; Load a value into DL
    mov dl, 071             ; Load the value 071 (ASCII code for 'q') into DL

    ; Check if DL is less than 'A'
    cmp dl, 'A'             ; Compare DL with ASCII value of 'A' (65)
    jl skip_display         ; Jump to 'skip_display' if DL < 'A'

    ; Check if DL is greater than 'Z'
    cmp dl, 'Z'             ; Compare DL with ASCII value of 'Z' (90)
    jg skip_display         ; Jump to 'skip_display' if DL > 'Z'

    ; If DL is between 'A' and 'Z', display the character
    mov ah, 2               ; DOS interrupt function to display a character
    int 21h                 ; Call interrupt 21h to display DL

skip_display:
```

```
    ; Exit the program
    mov ah, 4ch             ; DOS function to terminate the program
    int 21h                 ; Call interrupt to exit

main endp                   ; End of the main procedure
end main                    ; Mark the end of the program
```

Task – 4

```
.model small                ; Use the small memory model
.stack 100H                 ; Reserve 256 bytes for the stack
.data                       ; Data segment (not used in this example)
.code

main proc
    ; Initialize the data segment
    mov ax, @data           ; Load the address of the data segment
    mov ds, ax              ; Set the data segment register

; Initialize registers with values
mov ax, 4                   ; Set AX = 4
mov bx, 2                   ; Set BX = 2
mov cx, 6                   ; Set CX = 6

; First conditional check: Compare AX and BX
cmp ax, bx                  ; Compare AX with BX
jge skip_if_1               ; Jump to 'skip_if_1' if AX >= BX

; Second conditional check: Compare BX and CX
cmp bx, cx                  ; Compare BX with CX
jge skip_if_2               ; Jump to 'skip_if_2' if BX >= CX

; If AX < BX and BX < CX, execute this block
mov ax, 0                   ; Set AX = 0
jmp exit                    ; Skip the rest of the program and exit

skip_if_2:
    ; If BX >= CX, execute this block
    mov bx, 0               ; Set BX = 0
    jmp exit                ; Skip the rest of the program and exit

skip_if_1:
    ; If AX >= BX, execute this block
    jmp exit                ; Skip the rest of the program and exit

exit:
    ; Terminate the program
    mov ah, 4ch             ; DOS function to terminate the program
    int 21h                 ; Call interrupt to exit

main endp                   ; End of the main procedure
end main                    ; Mark the end of the program
```

Task – 5

```
.model small                ; Use the small memory model
.stack 100H                 ; Reserve 256 bytes for the stack
.data                       ; Data segment (not used in this example)
.code

main proc
    ; Initialize the data segment
    mov ax, @data           ; Load the address of the data segment
    mov ds, ax              ; Set the data segment register

    ; Initialize registers with values
    mov ax, 4               ; Set AX = 4
    mov bx, 2               ; Set BX = 2
    mov cx, 6               ; Set CX = 6

    ; First conditional check: Compare AX and BX
    cmp ax, bx              ; Compare AX with BX
    jge skip                ; Jump to 'skip' if AX >= BX

    ; If AX < BX, set DX to 0 and exit
    mov dx, 0               ; Set DX = 0
    jmp exit                ; Skip the rest of the program and exit

skip:
    ; Second conditional check: Compare BX and CX
    cmp bx, cx              ; Compare BX with CX
    jge set_one             ; Jump to 'set_one' if BX >= CX

    ; If BX < CX, set DX to 0 and exit
    mov dx, 0               ; Set DX = 0
    jmp exit                ; Skip the rest of the program and exit

set_one:
    ; If AX >= BX and BX >= CX, set DX to 1
    mov dx, 1               ; Set DX = 1

exit:
    ; Terminate the program
    mov ah, 4ch             ; DOS function to terminate the program
    int 21h                 ; Call interrupt to exit

main endp                   ; End of the main procedure
end main                    ; Mark the end of the program
```

Task – 6

```
.model small                ; Use the small memory model
.stack 100H                 ; Reserve 256 bytes for the stack
.data                       ; Data segment (not used in this example)
.code

main proc
```

```
; Initialize the data segment
mov ax, @data           ; Load the address of the data segment
mov ds, ax              ; Set the data segment register

; Initialize registers with values
mov ax, 4               ; Set AX = 4
mov bx, 2               ; Set BX = 2
mov cx, 6               ; Set CX = 6

; Compare AX and BX
cmp ax, bx              ; Compare AX with BX
jge check_bx_cx         ; Jump to 'check_bx_cx' if AX >= BX

; If AX < BX, set AX to 0 and exit
mov ax, 0               ; Set AX = 0
jmp exit                ; Skip the rest of the program and exit

check_bx_cx:
    ; Compare BX and CX
    cmp bx, cx              ; Compare BX with CX
    jge set_cx_zero         ; Jump to 'set_cx_zero' if BX >= CX

    ; If BX < CX, set BX to 0 and exit
    mov bx, 0               ; Set BX = 0
    jmp exit                ; Skip the rest of the program and exit

set_cx_zero:
    ; If BX >= CX, set CX to 0
    mov cx, 0               ; Set CX = 0

exit:
    ; Terminate the program
    mov ah, 4ch             ; DOS function to terminate the program
    int 21h                 ; Call interrupt to exit

main endp                   ; End of the main procedure
end main                    ; Mark the end of the program
```

Task – 7

```
.model small                ; Use the small memory model
.stack 100H                 ; Reserve 256 bytes for the stack
.data                       ; Data segment (not used in this example)
.code

main proc
    ; Initialize the data segment
    mov ax, @data           ; Load the address of the data segment
    mov ds, ax              ; Set the data segment register

    ; Initialize AL with a value
    mov al, 1               ; Set AL = 1 (example input)
```

```
    ; Compare AL with predefined values
    cmp al, 1               ; Check if AL = 1
    je display_o            ; Jump to 'display_o' if AL = 1
    cmp al, 3               ; Check if AL = 3
    je display_o            ; Jump to 'display_o' if AL = 3

    cmp al, 2               ; Check if AL = 2
    je display_e            ; Jump to 'display_e' if AL = 2
    cmp al, 4               ; Check if AL = 4
    je display_e            ; Jump to 'display_e' if AL = 4

    ; If AL doesn't match any value, exit the program
    jmp exit                ; Skip the rest of the program and exit

display_o:
    ; If AL is 1 or 3, prepare to display 'o'
    mov dl, 'o'             ; Set DL = 'o' (ASCII for 'o')
    jmp display_char        ; Jump to 'display_char' to output the character

display_e:
    ; If AL is 2 or 4, prepare to display 'e'
    mov dl, 'e'             ; Set DL = 'e' (ASCII for 'e')

display_char:
    ; Display the character stored in DL
    mov ah, 2               ; DOS function to display a character
    int 21h                 ; Call interrupt to display the character

exit:
    ; Terminate the program
    mov ah, 4ch             ; DOS function to terminate the program
    int 21h                 ; Call interrupt to exit

main endp                   ; End of the main procedure
end main                    ; Mark the end of the program
```